

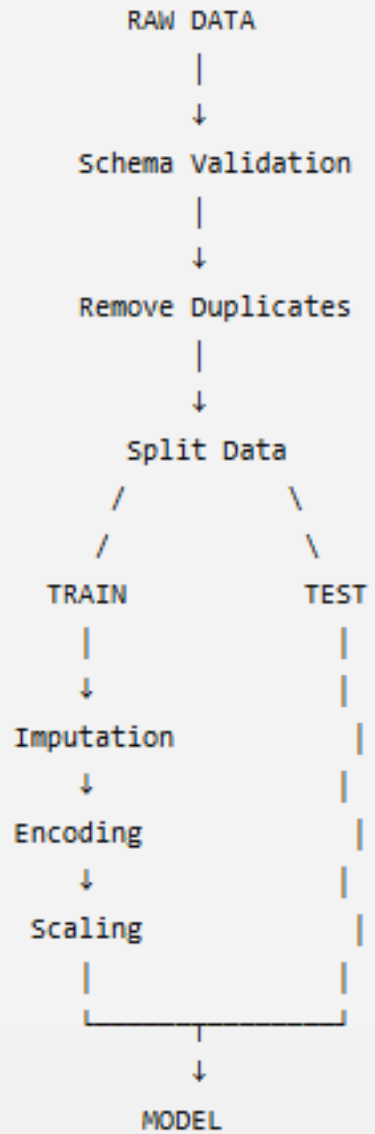
Machine Learning Diploma

Session 3: Data Pre-processing

Agenda:

0	Data Pre-processing
1	Data Understanding
2	Check for Datatypes
3	Handle Null Values
4	Handle Outliers
5	Remove Duplicates
6	Visualization
7	Data Splitting
8	Normalization
9	Encoding

Data Pre-processing



0. What is Data Pre-processing

What is Data Pre-processing:

- Data Pre-processing means applying different operations over the data to make it **clean**, **organized**, and **consistent**.
- Data can have many problems; such as, **Non-Informative data**, **In- consistent data**, **duplicated data**, etc.
- Data Pre-processing is about applying operations over the data to solve these problems.
- In this chapter, we will go through each of these problems and see how to solve it. Where we will apply all these concepts, in practice, on a dataset called **Titanic Dataset**.

Titanic Dataset:

- Titanic dataset is a data collected, about the individuals who were aboard the Titanic on its first and last voyage.
- This dataset was collected to study the behavior followed in rescuing people, to understand the reasons behind the survival of some categories of people and drowning of others.



Read Titanic Dataset:

- The first step in any project, is to read the csv/xlsx file that represents the Titanic dataset.
- As you can see the dataset has 12 rows (samples), and 891 columns (attributes).

```
1 df = pd.read_csv("train.csv")
2 df
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

1. Data Understanding

Data Understanding:

- Data Understanding means **studying and understanding** the data, including knowing:
 - What is the **task** or the **target** of the data.
 - How much **each column** is **important** to the **target**.
- For example, in **Titanic dataset**:
 - Our **task** is to know if a person **survived or died**, which means that the target is the column called “**Survived**”.
 - Columns like “**PassengerId**”, “**Name**”, “**Ticket**”, are **not important to the target**, so we will **drop these columns**.
- Note that all the columns except for the target columns are **called features**.

Drop Un-necessary Columns:

```
1 df.drop(["PassengerId", "Name", "Ticket"], axis=1, inplace = True)  
2 df.head()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
0	0	3	male	22.0	1	0	7.2500	NaN	S
1	1	1	female	38.0	1	0	71.2833	C85	C
2	1	3	female	26.0	0	0	7.9250	NaN	S
3	1	1	female	35.0	1	0	53.1000	C123	S
4	0	3	male	35.0	0	0	8.0500	NaN	S

2. Check for Datatypes

Check for Dtypes:

- Sometimes there are mistakes in representing the columns datatypes, for example:
 - A numerical column could be represented as categorical, or a categorical column represented as object datatype, and so on.
 - It's our responsibility to check for such mistakes and correct them.
- For example, in **Titanic dataset**:
 - Columns “**Survived**”, “**Pclass**”, “**SibSp**”, and “**Parch**” are int dtypes with small number of possible unique values, so we will **change them to be categorical**.
 - Columns “**Sex**”, and “**Embarked**” are string columns so we will **change them to be categorical**.

Change In-correct Datatypes:

- Display Datatypes

```
1 dtypes = df.dtypes
2 n_uniq = df.nunique()
3 pd.DataFrame({"Dtypes": dtypes, "Num_Unique": n_uniq}).T
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
Dtypes	int64	int64	object	float64	int64	int64	float64	object	object
Num_Unique	2	3	2	88	7	7	248	147	3

- Change In-correct Datatypes

```
1 cols = ["Pclass", "SibSp", "Parch", "Sex", "Embarked", "Survived"]
2 df[cols] = df[cols].astype('category')
3 pd.DataFrame(df.dtypes).T
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
0	category	category	category	float64	category	category	float64	object	category

Configuration tells the application what to do; code tells it how to do it

3. Handle Null Values

Null Values:

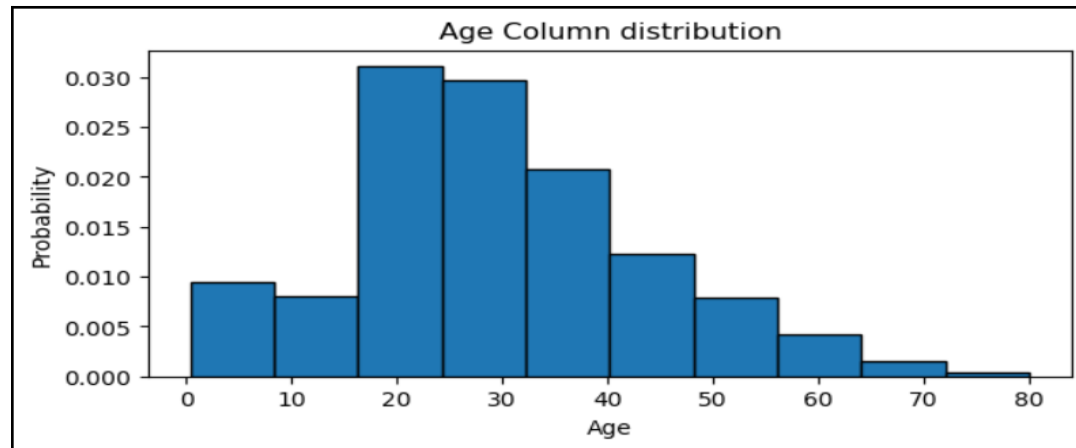
- **Null Values** must be handled, so that the data becomes complete and ready for machine learning.
- There are **3 options** to handle Null values, that **depend on the Null Values amount in each column**:
 - If the column contains a **small number of Null Values**, you can simply **delete rows that contain null values in this column**.
 - If the column contains a **huge number of Null Values**, you should **delete this column**.
 - If the column contains **a large number of Null Values, but not very large**, you can **replace null values with mean, median, or Mode**.

Replace Null Values:

- In the 3rd option:
 - We replace Null Values with **Mode** if the **column is categorical**.
 - We replace Null Values with **Mean** if the **column is numerical & normally distributed**.
 - We replace Null Values with **Median** if the **column is numerical & not-normally distributed**.

Check for Null Values:

- In Titanic dataset:
 - Embarked has only two null values, So we can just drop them.
 - Cabin has about 77% null values, So we will drop this column.
 - Age has about 20% null values, So we will replace null values with the median since Age is not-normally distributed (right skewed).



Handle Null Values:

Check for Null Values

```
1 null = df.isnull().sum()
2 ratio = null / df.shape[0]
3 pd.DataFrame({"Null_sum": null, "Ratio": ratio}).T
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
Null_sum	0.0	0.0	0.0	177.000000	0.0	0.0	0.0	687.000000	2.000000
Ratio	0.0	0.0	0.0	0.198653	0.0	0.0	0.0	0.771044	0.002245

Drop Null Values in Embarked Column

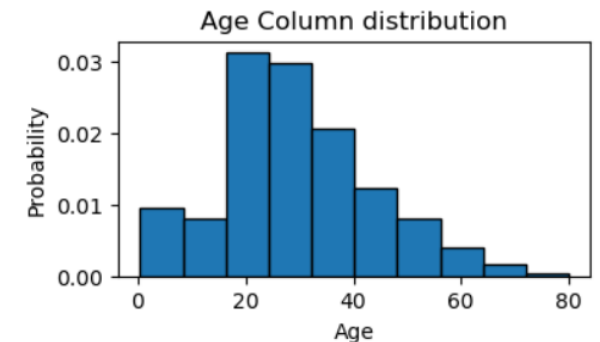
```
1 df = df.dropna(subset=['Embarked'])
```

Drop Cabin Column

```
1 df = df.drop("Cabin", axis=1)
```

Replace Null Values in Age Column

```
1 plt.figure(figsize=(4, 2))
2 plt.hist(df['Age'], density=True, edgecolor="black")
3 plt.title("Age Column distribution")
4 plt.xlabel("Age")
5 plt.ylabel("Probability")
6 plt.show()
```



```
1 median = df["Age"].median()
2 df["Age"].fillna(median, inplace=True)
```

Make sure Null Values are removed

```
1 pd.DataFrame(df.isnull().sum()).T
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	0	0	0	0	0	0	0	0

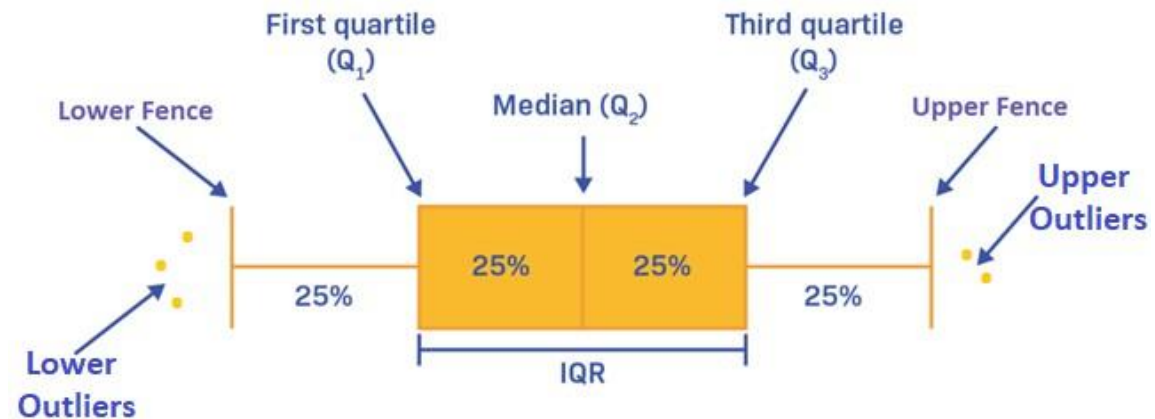
4. Handle Outliers

Outliers:

- Outliers are values within the data, that has **extremely large or extremely small values**, for example, people ages usually cannot exceed 100 years, if you find a 150 years-old man then this one is considered to be an outlier.
- Sometimes we need to **remove outliers** because they could cause many problems; such as, **introducing misleading information about the data**, for example there could 150 years-old man in real-life, but this is a very **special case that doesn't represent the normal human ages**.
- Also some **statistical measures could be sensitive to outliers**; such as **mean**, and this is another reason for why we would like to remove outliers.

Check for Outliers:

- To check for outliers we use Quartiles, where:
 - All values, **greater than the Upper-Fence**, are considered to be **outliers**.
 - All values, **smaller than the Lower-Fence**, are considered to be **outliers**.
- Also, there is a graph called boxplot, which is useful for quickly checking the outliers.



Handle Outliers:

- There are two options to handle outliers:
 - The 1st option is to treat outliers as if they were Null Values.
 - The 2nd option is to replace Upper-Outliers with Upper-Fence and replace Lower-Outliers with Lower-Fence.
- We Usually tend to do the Second option.
- In Titanic dataset, there are two columns that contains outliers;
 - “Age”, and “Fare”.

